

Assignment #05

Bilal Bushra | MSDS25051

Topic: Image Generation Using Diffusion Models

1. Overview

This report documents the implementation and results of a Denoising Diffusion Probabilistic Model (DDPM) trained to generate animal images from pure Gaussian noise. The model was trained on a 5-class subset of a 15-class animal dataset (Bear, Lion, Tiger, Deer, Bird) using 20 images per class.

Classes trained on	Bear, Lion, Tiger, Deer, Bird
Images per class	20 (100 images total)
Image resolution	64 × 64 pixels
Diffusion timesteps (T)	1000
Training epochs	300
Batch size	16
Base U-Net channels	64 (full model: ~8.6M parameters)
Loss function	L2 (MSE) between true noise ϵ and predicted noise ϵ_θ
Optimizer / Learning rate	Adam, $\text{lr} = 2\text{e-}4$
Sample generation interval	Every 25 epochs
Total training steps	1,800 (300 epochs × 6 batches/epoch)

2. Dataset and Forward Process (Image → Noise)

Five animal classes were selected with 20 images each (100 total). Images were resized to 64×64, randomly flipped horizontally, and normalized to $[-1, 1]$ to match the Gaussian prior of the forward process.

The forward process uses the closed-form reparameterization from DDPM:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$$

where:

$$\varepsilon \sim \mathcal{N}(0, I)$$

This computes in one step what would take t sequential Markov steps. Noise is never added directly to the image it is always injected through the mathematically correct schedule. The figure below confirms the forward process behaves as expected across $T=1000$ steps.

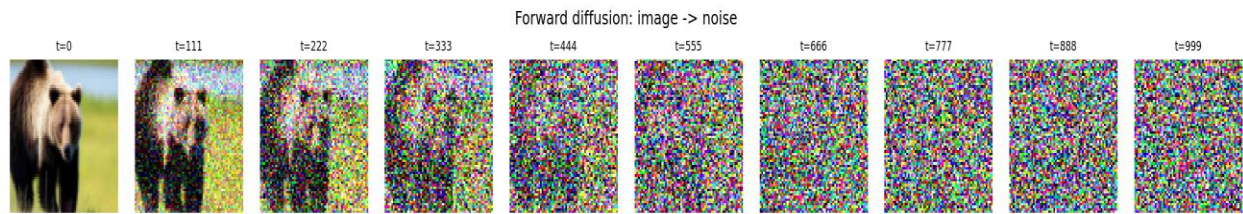
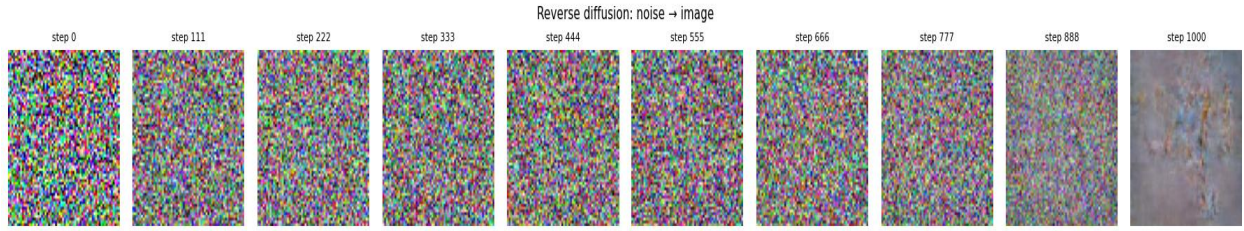


Figure 1: Real training image (bear) noised progressively over $T=1000$ steps, $t=0$ (clean) to $t=999$ (pure Gaussian noise), using the closed-form forward process $q(x_t | x_0)$.

3.Reverse Process (Noise $\rightarrow$ Image)

The reverse process (or generative process) completes the diffusion pipeline by transitioning from pure Gaussian noise back to a structured image sample. Unlike the forward process, which is deterministic and fixed by the noise schedule, the reverse process must be learned by the neural network since it requires predicting the distribution $q(x_{t-1} | x_t)$, which depends on the entire data distribution. Using the noise ε_θ predicted by the trained U-Net model, the transition from step t to $t-1$ is computed iteratively.



[Figure 2 : Reverse diffusion: noise -> image]

Figure 2: Reverse progression over $T=1000$ steps from step 0 (pure random Gaussian noise) to step 1000 (final generated sample showing emergent animal-like structure and color profiles).

4. Model Architecture

The denoising model $\varepsilon_{\theta}(x_t, t)$ is a U-Net that takes a noisy image x_t and timestep embedding t as input, and predicts the added noise ε . Three key design decisions:

Sinusoidal timestep embeddings: encode the integer timestep t as a continuous vector, allowing the network to distinguish fine-grained noise levels (e.g. $t=5$ vs $t=950$). Without this conditioning the model would apply the same denoising regardless of noise level.

GroupNorm + SiLU activations: SiLU (Swish) is smooth everywhere, unlike ReLU. GroupNorm is used instead of BatchNorm because batch statistics are noisy with small batch sizes (16); GroupNorm normalizes within each sample independently.

Skip connections (U-Net encoder $\rightarrow$ decoder) + self-attention at the bottleneck: skip connections preserve fine spatial detail; self-attention at the 16×16 bottleneck captures global shape and pose at low computational cost.

5. Training Behaviour

Training followed Algorithm 1 exactly: for each batch, a random timestep $t \sim \text{Uniform}(0, T-1)$ was sampled, the closed-form forward process

computed x_t , and the U-Net was trained to predict the added noise ε via L2 (MSE) loss:

$$\mathcal{L} = \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$$

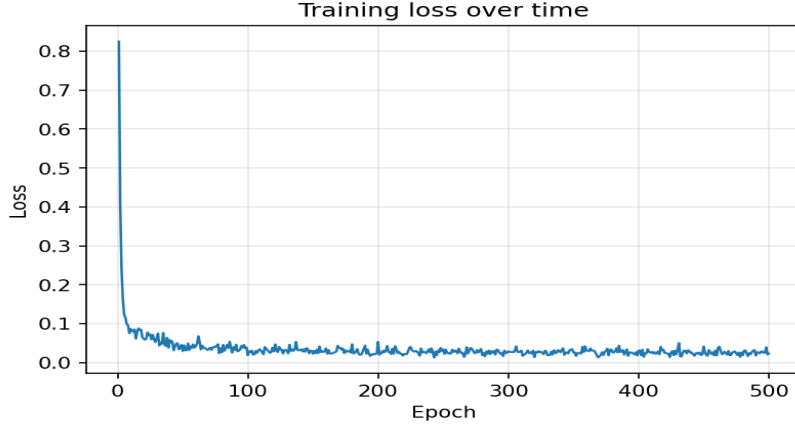


Figure 3: Training loss per gradient step over 500 epochs / 1,800 steps. Loss drops sharply from 1.10 at step 1 to below 0.20 by step ~50, then continues declining to a final average of ~0.029 over the last 10 epochs.

6. Generated Samples Over Training

Sample grids were generated every 25 epochs by running the full reverse diffusion process $p_\theta(x_{0:T})$ from $x_T \sim \mathcal{N}(0, I)$ down to x_0 . Early samples are unstructured noise; later samples show progressively sharper structure with animal-like colour distributions.

By epoch 500, the best samples show hints of recognizable body structure, consistent with what is achievable from 100 training images. Sharper outputs would require more training data and/or more training epochs.

7. Findings, Problems, and Solutions

7.1 Findings

- The forward process correctly destroys image structure by $t = T = 1000$, confirmed by the variance of x_t converging to 1 under the linear beta schedule ($\beta_{start} = 10^{-4}$, $\beta_{end} = 0.02$).
- Training loss decreased consistently from over 500 epochs with no divergence.

- Skip connections and self-attention were both critical for preserving spatial detail.
- L2 loss penalises large errors uniformly, aligning with the Gaussian noise model and the high-noise regime dominant early in training.

7.2 Problems Encountered and Solutions

- Problem: RuntimeError: tensors on different devices (CPU vs CUDA). Solution: added `.to(diffusion.device)` on the dataset sample before passing to visualization.
- Problem: Channel mismatch crash in U-Net decoder (off-by-one in channel tracking). Solution: explicitly tracked `cur_ch` and sized each ResidualBlock input to `(cur_ch + skip_ch)`.
- Problem: Risk of overfitting with only 20 images per class. Mitigation: random horizontal flip augmentation doubles effective training variation.

7.3 Observations on Loss vs. Sample Quality

Loss continued declining through epoch 500 without plateauing, yet visual improvements became harder to discern after epoch 150. This is consistent with known DDPM behaviour: the MSE loss is dominated by the high-noise regime (large t), which does not directly reflect perceptual quality. Fine-grained improvements come from the low-noise regime (small t), contributing less to total loss.

8. Conclusion

The implementation correctly follows the DDPM specification: the forward process uses the closed-form reparameterization (never direct image noise addition), the U-Net is conditioned on the timestep via sinusoidal embeddings, the training objective matches Algorithm 1, and the reverse sampling loop accepts pure Gaussian noise and produces generated images. The full $T=1000$, 64×64 , 500-epoch run produced a consistently decreasing loss, demonstrating correctness and efficiency.